

PENGUMPULAN TAHAP 3

Nama: Robani Diansyah

NPM: 23312246

ORDER-SERVICE

1. api/services/order-service/prisma/schema.prisma

```
enum OrderStatus {
  PENDING
  PAID
  CANCELLED
}

model Order {
  id          Int          @id @default(autoincrement())
  userId      Int
  status      OrderStatus @default(PENDING)
  totalHarga  Int
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  items       OrderItem[]
}

model OrderItem {
  id          Int          @id @default(autoincrement())
  orderId     Int
  productId   Int
  jumlah      Int
  harga       Int
  createdAt   DateTime     @default(now())
  order       Order        @relation(fields: [orderId], references: [id])
}
```

File ini mendefinisikan struktur tabel Order dan OrderItem di database. Order menyimpan data pesanan utama, sedangkan OrderItem menyimpan detail produk yang dibeli dalam satu pesanan.

Field	Tipe	Fungsi
id	Int	Primary key, otomatis bertambah setiap ada data baru
userId	Int	ID user yang membuat pesanan, mengacu ke auth-service (tanpa foreign key, karena beda service)
status	OrderStatus (enum)	Status pesanan: PENDING, PAID, atau CANCELLED. Pakai enum biar type-safe dan tidak typo

totalHarga	Int	Total harga keseluruhan pesanan, dihitung dari semua item
createdAt	DateTime	Waktu pesanan dibuat, otomatis diisi Prisma
updatedAt	DateTime	Waktu data terakhir diubah, otomatis diperbarui Prisma
items	OrderItem[]	Relasi 1 ke banyak ke tabel OrderItem
orderId	Int	ID order yang menjadi induk dari item ini
productId	Int	ID produk yang dibeli, mengacu ke product-service (tanpa foreign key)
jumlah	Int	Jumlah produk yang dibeli
harga	Int	Harga satuan produk saat dibeli, disimpan sendiri agar histori transaksi tidak berubah walau harga produk di kemudian hari berubah

File main.ts, app.module.ts, prisma.module.ts, dan prisma.service.ts sama persis dengan kategori-service, hanya beda nama module dan port (order-service pakai port 3004).

2. api/services/order-service/src/order/dto/create-order.dto.ts

```
import { IsInt, IsArray, ArrayMinSize, Min, ValidateNested } from 'class-validator';
import { Type } from 'class-transformer';

// DTO untuk validasi item yang dibeli
export class CreateOrderItemDto {
  @IsInt({ message: 'Product ID harus berupa angka' })
  productId!: number;

  @IsInt({ message: 'Jumlah harus berupa angka' })
  @Min(1, { message: 'Jumlah minimal 1' })
  jumlah!: number;

  @IsInt({ message: 'Harga harus berupa angka' })
  @Min(0, { message: 'Harga tidak boleh negatif' })
  harga!: number;
}

// DTO untuk validasi request body saat create order
export class CreateOrderDto {
  @IsInt({ message: 'User ID harus berupa angka' })
  userId!: number;

  @IsArray({ message: 'Items harus berupa array' })
  @ArrayMinSize(1, { message: 'Order minimal memiliki 1 item' })
  @ValidateNested({ each: true })
  @Type(() => CreateOrderItemDto)
  items!: CreateOrderItemDto[];
}
```

File ini mendefinisikan struktur dan aturan validasi data saat membuat order baru. Ada 2 class karena order terdiri dari data utama (userId) dan array item yang dibeli.

Bagian	Fungsi
CreateOrderItemDto	Class validasi untuk setiap item produk yang dibeli dalam order
productId, jumlah, harga	Wajib diisi dan harus berupa angka. jumlah minimal 1, harga tidak boleh negatif
CreateOrderDto	Class validasi untuk request body utama saat create order
userId	Wajib diisi dan harus berupa angka, menandakan siapa yang melakukan pesanan
@IsArray()	Memastikan items berupa array, bukan tipe data lain
@ArrayMinSize(1)	Order minimal harus memiliki 1 item, tidak boleh kosong
@ValidateNested({ each: true })	Setiap item di dalam array divalidasi juga menggunakan aturan dari CreateOrderItemDto
@Type(() => CreateOrderItemDto)	Mengubah setiap objek di array items menjadi instance CreateOrderItemDto agar validasi nested berjalan

3. api/services/order-service/src/order/dto/update-order.dto.ts

```
import { IsEnum } from 'class-validator';
import { OrderStatus } from '../../../generated/prisma/client';

// DTO untuk update status order
export class UpdateOrderDto {
  @IsEnum(OrderStatus, { message: 'Status tidak valid' })
  status!: OrderStatus;
}
```

Berbeda dengan service lain, update-order.dto.ts tidak meng-extend dari CreateOrderDto karena update order hanya untuk mengubah status, bukan mengubah seluruh data pesanan.

Bagian	Fungsi
UpdateOrderDto	Class validasi khusus untuk request body saat update status order
@IsEnum(OrderStatus)	Memastikan status yang dikirim sesuai dengan enum OrderStatus (PENDING, PAID, CANCELLED), kalau tidak sesuai langsung error
status!: OrderStatus	Field wajib diisi dan tipenya diambil langsung dari enum hasil generate Prisma, jadi type-safe

4. api/services/order-service/src/order/entities/order.entity.ts

```
import { OrderStatus } from '../../../generated/prisma/client';

// Representasi item dalam order
export class OrderItem {
  id!: number;
  orderId!: number;
  productId!: number;
  jumlah!: number;
  harga!: number;
  createdAt!: Date;
}

// Representasi model Order sesuai schema Prisma
export class Order {
  id!: number;
  userId!: number;
  status!: OrderStatus;
  totalHarga!: number;
  createdAt!: Date;
  updatedAt!: Date;
  items!: OrderItem[];
}
```

File ini mendefinisikan struktur data Order dan OrderItem yang merepresentasikan model di database, dipakai sebagai tipe return di controller dan service.

Bagian	Fungsi
class OrderItem	Representasi satu item produk dalam sebuah order
id, orderId, productId, jumlah, harga, createdAt	Field item sesuai schema Prisma, semua wajib ada
class Order	Representasi data order utama
status!: OrderStatus	Tipe status diambil dari enum hasil generate Prisma, jadi otomatis hanya boleh PENDING, PAID, atau CANCELLED
items!: OrderItem[]	Array berisi semua item yang dibeli dalam order ini

5. api/services/order-service/src/order/order.service.ts

```
import { Injectable, NotFoundException } from '@nestjs/common';
import { PrismaService } from '../../../prisma/prisma.service';
import { CreateOrderDto } from '../../../dto/create-order.dto';
import { UpdateOrderDto } from '../../../dto/update-order.dto';
import { Order, OrderItem } from '../../../generated/prisma/client';

// Tipe return untuk order beserta itemnya
type OrderWithItems = Order & { items: OrderItem[] };

@Injectable()
export class OrderService {
  // Inject PrismaService untuk akses database
```

```

constructor(private readonly prisma: PrismaService) {}

// Buat order baru beserta item-itemnya
async create(createOrderDto: CreateOrderDto): Promise<OrderWithItems> {
  // Hitung total harga dari semua item
  const totalHarga = createOrderDto.items.reduce(
    (acc, item) => acc + item.harga * item.jumlah,
    0,
  );

  return this.prisma.order.create({
    data: {
      userId: createOrderDto.userId,
      totalHarga,
      items: {
        create: createOrderDto.items,
      },
    },
    include: { items: true },
  });
}

// Ambil semua order beserta itemnya
async findAll(): Promise<OrderWithItems[]> {
  return this.prisma.order.findMany({
    orderBy: { createdAt: 'desc' },
    include: { items: true },
  });
}

// Ambil satu order berdasarkan id
async findOne(id: number): Promise<OrderWithItems> {
  const order = await this.prisma.order.findUnique({
    where: { id },
    include: { items: true },
  });

  if (!order) {
    throw new NotFoundException(`Order dengan id ${id} tidak ditemukan`);
  }

  return order;
}

// Update status order berdasarkan id
async update(
  id: number,
  updateOrderDto: UpdateOrderDto,
): Promise<OrderWithItems> {
  await this.findOne(id);

  return this.prisma.order.update({
    where: { id },
    data: { status: updateOrderDto.status },
    include: { items: true },
  });
}

// Hapus order berdasarkan id

```

```

async remove(id: number): Promise<OrderWithItems> {
  await this.findOne(id);

  await this.prisma.orderItem.deleteMany({
    where: { orderId: id },
  });

  return this.prisma.order.delete({
    where: { id },
    include: { items: true },
  });
}
}

```

File ini berisi semua logic bisnis order. Bedanya dengan kategori/product, ada perhitungan total harga otomatis dan relasi dengan tabel OrderItem.

Bagian	Fungsi
OrderWithItems	Tipe gabungan Order dan array OrderItem, dipakai sebagai return type semua method
create()	Hitung totalHarga dari seluruh item (harga x jumlah), lalu simpan order beserta seluruh item-nya sekaligus menggunakan nested create
findAll()	Ambil semua order, diurutkan dari yang terbaru, sekaligus include data items
findOne()	Ambil satu order berdasarkan id beserta items-nya, lempar 404 kalau tidak ketemu
update()	Cek order ada dulu, lalu update field status saja sesuai UpdateOrderDto
remove()	Cek order ada dulu, hapus semua OrderItem terkait lebih dulu, baru hapus data Order-nya
include: { items: true }	Setiap query order akan otomatis menyertakan data item-item di dalamnya

6. api/services/order-service/src/order/order.controller.ts

```

import {
  Controller, Get, Post, Body, Patch, Param, Delete,
  HttpStatusCode, HttpStatus, ParseIntPipe,
} from '@nestjs/common';
import { OrderService } from '../order.service';
import { CreateOrderDto } from '../dto/create-order.dto';
import { UpdateOrderDto } from '../dto/update-order.dto';
import { Order, OrderItem } from '../generated/prisma/client';

type OrderWithItems = Order & { items: OrderItem[] };

```

```
// Semua route di sini -> /api/order
@Controller('order')
export class OrderController {
  constructor(private readonly orderService: OrderService) {}

  @Post()
  @HttpCode(HttpStatus.CREATED)
  async create(@Body() createOrderDto: CreateOrderDto): Promise<OrderWithItems> {
    return this.orderService.create(createOrderDto);
  }

  @Get()
  async findAll(): Promise<OrderWithItems[]> {
    return this.orderService.findAll();
  }

  @Get('/:id')
  async findOne(@Param('id', ParseIntPipe) id: number): Promise<OrderWithItems> {
    return this.orderService.findOne(id);
  }

  @Patch('/:id')
  async update(
    @Param('id', ParseIntPipe) id: number,
    @Body() updateOrderDto: UpdateOrderDto,
  ): Promise<OrderWithItems> {
    return this.orderService.update(id, updateOrderDto);
  }

  @Delete('/:id')
  @HttpCode(HttpStatus.OK)
  async remove(@Param('id', ParseIntPipe) id: number): Promise<OrderWithItems> {
    return this.orderService.remove(id);
  }
}
```

Pola dan struktur sama seperti kategori.controller.ts, yang beda hanya route (/api/order) dan return type yang menyertakan items.

Bagian	Fungsi
@Controller('order')	Semua endpoint di file ini otomatis diawali /api/order
POST /api/order	Membuat order baru beserta item-itemnya, response 201
GET /api/order	Mengambil semua order beserta items
GET /api/order/:id	Mengambil satu order berdasarkan id beserta items
PATCH /api/order/:id	Mengubah status order berdasarkan id
DELETE /api/order/:id	Menghapus order beserta seluruh item-nya, response 200

INVENTORY-SERVICE

1. api/services/inventory-service/prisma/schema.prisma

```
enum StockLogType {
  IN
  OUT
}

model Inventory {
  id          Int      @id @default(autoincrement())
  productId   Int      @unique
  stock       Int      @default(0)
  updatedAt   DateTime @updatedAt
}

model StockLog {
  id          Int      @id @default(autoincrement())
  productId   Int
  quantity    Int
  type        StockLogType
  note        String?
  createdAt   DateTime @default(now())
}
```

File ini mendefinisikan struktur tabel Inventory dan StockLog. Inventory menyimpan stok terkini per produk, sedangkan StockLog menyimpan histori setiap perubahan stok untuk keperluan audit.

Field	Tipe	Fungsi
id (Inventory)	Int	Primary key, otomatis bertambah
productId (Inventory)	Int @unique	ID produk dari product-service, dibuat unik karena 1 produk hanya punya 1 record inventory
stock	Int	Jumlah stok terkini, default 0. Inilah sumber utama data stok, bukan dihitung dari log
updatedAt	DateTime	Waktu stok terakhir diubah, otomatis diperbarui Prisma
id (StockLog)	Int	Primary key untuk setiap catatan log
productId (StockLog)	Int	ID produk yang stoknya berubah
quantity	Int	Jumlah perubahan stok pada transaksi ini
type	StockLogType (enum)	Jenis perubahan: IN (stok masuk/restock) atau OUT (stok keluar/order)
note	String?	Catatan opsional, misal 'Restock dari petani' atau 'Order #1'
createdAt	DateTime	Waktu log dibuat, otomatis diisi Prisma

File main.ts, app.module.ts, prisma.module.ts, dan prisma.service.ts sama persis dengan kategori-service, hanya beda nama module dan port (inventory-service pakai port 3005).

2. api/services/inventory-service/src/inventory/dto/create-inventory.dto.ts

```
import { IsInt, Min } from 'class-validator';

// DTO untuk membuat record inventory baru untuk sebuah product
export class CreateInventoryDto {
  @IsInt({ message: 'Product ID harus berupa angka' })
  productId!: number;

  @IsInt({ message: 'Stock harus berupa angka' })
  @Min(0, { message: 'Stock tidak boleh negatif' })
  stock!: number;
}
```

File ini mendefinisikan validasi data saat pertama kali membuat record inventory untuk sebuah produk (biasanya saat produk baru ditambahkan).

Bagian	Fungsi
productId	Wajib diisi dan harus berupa angka, menandakan produk mana yang stoknya dicatat
stock	Wajib diisi, harus berupa angka, dan tidak boleh negatif. Ini adalah stok awal produk

3. api/services/inventory-service/src/inventory/dto/update-inventory.dto.ts

```
import { IsInt, IsEnum, IsOptional, IsString, Min } from 'class-validator';
import { StockLogType } from '../../generated/prisma/client';

// DTO untuk menambah atau mengurangi stok
export class UpdateInventoryDto {
  @IsInt({ message: 'Quantity harus berupa angka' })
  @Min(1, { message: 'Quantity minimal 1' })
  quantity!: number;

  @IsEnum(StockLogType, { message: 'Type harus IN atau OUT' })
  type!: StockLogType;

  @IsOptional()
  @IsString()
  note?: string;
}
```

Berbeda dengan service lain, file ini bukan untuk update data biasa, tapi khusus untuk menambah (restock) atau mengurangi (order) stok suatu produk.

Bagian	Fungsi
--------	--------

quantity	Wajib diisi, harus berupa angka, minimal 1. Ini adalah jumlah stok yang ditambah/dikurangi
@IsEnum(StockLogType)	Memastikan type yang dikirim hanya boleh IN atau OUT sesuai enum StockLogType
note	Opsional, catatan tambahan untuk histori, misal alasan perubahan stok

4. api/services/inventory-service/src/inventory/entities/inventory.entity.ts

```
import { StockLogType } from '../../../generated/prisma/client';

// Representasi model Inventory sesuai schema Prisma
export class Inventory {
  id!: number;
  productId!: number;
  stock!: number;
  updatedAt!: Date;
}

// Representasi model StockLog sesuai schema Prisma
export class StockLog {
  id!: number;
  productId!: number;
  quantity!: number;
  type!: StockLogType;
  note!: string | null;
  createdAt!: Date;
}
```

File ini mendefinisikan struktur data Inventory dan StockLog sesuai model Prisma, dipakai sebagai tipe return di controller dan service.

Bagian	Fungsi
class Inventory	Representasi data stok terkini sebuah produk
class StockLog	Representasi satu baris histori perubahan stok
type!: StockLogType	Tipe diambil dari enum hasil generate Prisma, otomatis hanya boleh IN atau OUT
note!: string null	Catatan log bisa null kalau memang tidak diisi saat adjust stok

5. api/services/inventory-service/src/inventory/inventory.service.ts

```
import { Injectable, NotFoundException, ConflictException } from '@nestjs/common';
import { PrismaService } from '../../../prisma/prisma.service';
import { CreateInventoryDto } from '../../../dto/create-inventory.dto';
import { UpdateInventoryDto } from '../../../dto/update-inventory.dto';
import { Inventory, StockLog, StockLogType } from '../../../generated/prisma/client';
```

```

@Injectable()
export class InventoryService {
  constructor(private readonly prisma: PrismaService) {}

  // Buat record inventory baru untuk sebuah product
  async create(createInventoryDto: CreateInventoryDto): Promise<Inventory> {
    const existing = await this.prisma.inventory.findUnique({
      where: { productId: createInventoryDto.productId },
    });

    if (existing) {
      throw new ConflictException(
        `Inventory untuk product id ${createInventoryDto.productId} sudah ada`,
      );
    }

    return this.prisma.inventory.create({
      data: createInventoryDto,
    });
  }

  // Ambil semua data inventory
  async findAll(): Promise<Inventory[]> {
    return this.prisma.inventory.findMany();
  }

  // Ambil inventory berdasarkan productId
  async findOne(productId: number): Promise<Inventory> {
    const inventory = await this.prisma.inventory.findUnique({
      where: { productId },
    });

    if (!inventory) {
      throw new NotFoundException(
        `Inventory untuk product id ${productId} tidak ditemukan`,
      );
    }

    return inventory;
  }

  // Tambah atau kurangi stok, sekaligus catat ke stock log
  async adjustStock(
    productId: number,
    updateInventoryDto: UpdateInventoryDto,
  ): Promise<Inventory> {
    const inventory = await this.findOne(productId);

    const newStock =
      updateInventoryDto.type === StockLogType.IN
        ? inventory.stock + updateInventoryDto.quantity
        : inventory.stock - updateInventoryDto.quantity;

    if (newStock < 0) {
      throw new ConflictException('Stok tidak cukup untuk dikurangi');
    }

    const [updatedInventory] = await this.prisma.$transaction([

```

```

    this.prisma.inventory.update({
      where: { productId },
      data: { stock: newStock },
    }),
    this.prisma.stockLog.create({
      data: {
        productId,
        quantity: updateInventoryDto.quantity,
        type: updateInventoryDto.type,
        note: updateInventoryDto.note,
      },
    }),
  ]);

  return updatedInventory;
}

// Ambil riwayat stock log berdasarkan productId
async findLogs(productId: number): Promise<StockLog[]> {
  return this.prisma.stockLog.findMany({
    where: { productId },
    orderBy: { createdAt: 'desc' },
  });
}
}

```

File ini berisi logic bisnis inventory. Yang membedakan dari service lain adalah adanya method `adjustStock` yang mengubah stok dan menulis log dalam satu transaction, serta validasi agar stok tidak menjadi negatif.

Bagian	Fungsi
<code>create()</code>	Cek dulu apakah produk sudah punya record inventory (productId unik), kalau sudah ada lempar 409 Conflict, kalau belum baru dibuat
<code>findAll()</code>	Ambil semua data inventory dari seluruh produk
<code>findOne()</code>	Ambil data inventory berdasarkan productId, lempar 404 kalau belum ada
<code>adjustStock()</code>	Hitung stok baru berdasarkan type IN (ditambah) atau OUT (dikurangi). Kalau hasil pengurangan jadi negatif, lempar 409 Conflict
<code>\$transaction([...])</code>	Update stok dan insert StockLog dijalankan bersamaan dalam satu transaction, supaya data konsisten -- kalau salah satu gagal, semua dibatalkan
<code>findLogs()</code>	Ambil semua histori perubahan stok untuk satu produk, diurutkan dari yang terbaru

6. `api/services/inventory-service/src/inventory/inventory.controller.ts`

```
import {
```

```

    Controller, Get, Post, Body, Patch, Param,
    HttpStatusCode, HttpStatus, ParseIntPipe,
  } from '@nestjs/common';
import { InventoryService } from '../inventory.service';
import { CreateInventoryDto } from '../dto/create-inventory.dto';
import { UpdateInventoryDto } from '../dto/update-inventory.dto';
import { Inventory, StockLog } from '../../generated/prisma/client';

// Semua route di sini -> /api/inventory
@Controller('inventory')
export class InventoryController {
  constructor(private readonly inventoryService: InventoryService) {}

  @Post()
  @HttpCode(HttpStatus.CREATED)
  async create(@Body() createInventoryDto: CreateInventoryDto): Promise<Inventory> {
    return this.inventoryService.create(createInventoryDto);
  }

  @Get()
  async findAll(): Promise<Inventory[]> {
    return this.inventoryService.findAll();
  }

  @Get(':productId')
  async findOne(@Param('productId', ParseIntPipe) productId: number): Promise<Inventory> {
    return this.inventoryService.findOne(productId);
  }

  // Tambah atau kurangi stok
  @Patch(':productId')
  async adjustStock(
    @Param('productId', ParseIntPipe) productId: number,
    @Body() updateInventoryDto: UpdateInventoryDto,
  ): Promise<Inventory> {
    return this.inventoryService.adjustStock(productId, updateInventoryDto);
  }

  // Riwayat perubahan stok
  @Get(':productId/logs')
  async findLogs(@Param('productId', ParseIntPipe) productId: number): Promise<StockLog[]> {
    return this.inventoryService.findLogs(productId);
  }
}

```

Pola dasar sama seperti controller lain, tapi endpoint `:productId` dipakai sebagai identifier (bukan id biasa) karena 1 produk = 1 record inventory, dan ada tambahan endpoint khusus untuk riwayat log.

Bagian	Fungsi
@Controller('inventory')	Semua endpoint di file ini otomatis diawali /api/inventory
POST /api/inventory	Membuat record inventory baru untuk sebuah produk, response 201

GET /api/inventory	Mengambil semua data inventory
GET /api/inventory/:productId	Mengambil data inventory satu produk berdasarkan productId
PATCH /api/inventory/:productId	Menambah (IN) atau mengurangi (OUT) stok produk, sekaligus mencatat log
GET /api/inventory/:productId/logs	Mengambil seluruh riwayat perubahan stok untuk satu produk

API-GATEWAY

Gateway adalah pintu masuk utama (port 3000) yang meneruskan (proxy) semua request dari user ke masing-masing microservice (kategori, product, auth, order, inventory). Polanya sama untuk semua module: service melakukan HTTP call ke service asli menggunakan `HttpService` dari `@nestjs/axios`, dan controller hanya meneruskan request ke service tersebut.

1. api/gateway/.env

```
KATEGORI_SERVICE_URL="http://localhost:3001/api"
PRODUCT_SERVICE_URL="http://localhost:3002/api"
AUTH_SERVICE_URL="http://localhost:3003/api"
ORDER_SERVICE_URL="http://localhost:3004/api"
INVENTORY_SERVICE_URL="http://localhost:3005/api"
```

File ini menyimpan daftar alamat (base URL) dari setiap microservice. Gateway membaca variabel ini untuk tahu kemana request harus diteruskan, sehingga alamat service tidak hardcode di dalam kode.

Bagian	Fungsi
KATEGORI_SERVICE_URL	Alamat kategori-service (port 3001)
PRODUCT_SERVICE_URL	Alamat product-service (port 3002)
AUTH_SERVICE_URL	Alamat auth-service (port 3003)
ORDER_SERVICE_URL	Alamat order-service (port 3004)
INVENTORY_SERVICE_URL	Alamat inventory-service (port 3005)

2. api/gateway/src/kategori/kategori.service.ts

```
import { HttpService } from '@nestjs/axios';
import { Injectable } from '@nestjs/common';
import { firstValueFrom } from 'rxjs';

@Injectable()
export class KategoriService {
  // URL kategori-service diambil dari .env
```

```

private readonly baseUrl = process.env['KATEGORI_SERVICE_URL'];

constructor(private readonly httpService: HttpService) {}

async findAll(): Promise<unknown> {
  const response = await firstValueFrom(
    this.httpService.get(`${this.baseUrl}/kategori`),
  );
  return response.data;
}

async findOne(id: number): Promise<unknown> {
  const response = await firstValueFrom(
    this.httpService.get(`${this.baseUrl}/kategori/${id}`),
  );
  return response.data;
}

async create(data: unknown): Promise<unknown> {
  const response = await firstValueFrom(
    this.httpService.post(`${this.baseUrl}/kategori`, data),
  );
  return response.data;
}

async update(id: number, data: unknown): Promise<unknown> {
  const response = await firstValueFrom(
    this.httpService.patch(`${this.baseUrl}/kategori/${id}`, data),
  );
  return response.data;
}

async remove(id: number): Promise<unknown> {
  const response = await firstValueFrom(
    this.httpService.delete(`${this.baseUrl}/kategori/${id}`),
  );
  return response.data;
}
}

```

File ini adalah proxy service untuk kategori. Tugasnya hanya meneruskan request ke kategori-service menggunakan HttpService, tidak ada logic bisnis sama sekali. Pola yang sama persis dipakai juga untuk product, order, dan inventory (hanya beda baseUrl dan path endpoint).

Bagian	Fungsi
baseUrl	Alamat kategori-service diambil dari .env melalui process.env, tidak hardcode
HttpService	Service dari @nestjs/axios untuk melakukan HTTP request ke service lain
firstValueFrom()	Mengubah response HttpService (berbentuk Observable) menjadi Promise agar bisa di-await

findAll(), findOne(), create(), update(), remove()	Masing-masing meneruskan request GET, GET by id, POST, PATCH, DELETE ke kategori-service dan mengembalikan response.data apa adanya
Promise<unknown>	Tipe return generic karena response bentuknya tergantung dari service tujuan

File product.service.ts, order.service.ts, dan inventory.service.ts memiliki struktur yang sama persis, hanya beda nama class, baseUrl (mengambil env masing-masing), dan path endpoint (/product, /order, /inventory).

3. api/gateway/src/auth/auth.service.ts

```
import { HttpService } from '@nestjs/axios';
import { Injectable } from '@nestjs/common';
import { firstValueFrom } from 'rxjs';

@Injectable()
export class AuthService {
  private readonly baseUrl = process.env['AUTH_SERVICE_URL'];

  constructor(private readonly httpService: HttpService) {}

  async register(data: unknown): Promise<unknown> {
    const response = await firstValueFrom(
      this.httpService.post(`${this.baseUrl}/auth/register`, data),
    );
    return response.data;
  }

  async login(data: unknown): Promise<unknown> {
    const response = await firstValueFrom(
      this.httpService.post(`${this.baseUrl}/auth/login`, data),
    );
    return response.data;
  }
}
```

Berbeda dengan service lain yang berbentuk CRUD, auth.service.ts hanya punya 2 method karena auth-service memang hanya menyediakan endpoint register dan login.

Bagian	Fungsi
register()	Meneruskan request POST /auth/register ke auth-service
login()	Meneruskan request POST /auth/login ke auth-service

4. api/gateway/src/kategori/kategori.controller.ts

```
import {
  Controller, Get, Post, Body, Patch, Param, Delete, ParseIntPipe,
} from '@nestjs/common';
import { KategoriService } from '../kategori.service';
```



```
// Semua route di sini -> /api/kategori (di-forward ke kategori-service)
@Controller('kategori')
export class KategoriController {
  constructor(private readonly kategoriService: KategoriService) {}

  @Get()
  async findAll(): Promise<unknown> {
    return this.kategoriService.findAll();
  }

  @Get('/:id')
  async findOne(@Param('id', ParseIntPipe) id: number): Promise<unknown> {
    return this.kategoriService.findOne(id);
  }

  @Post()
  async create(@Body() data: unknown): Promise<unknown> {
    return this.kategoriService.create(data);
  }

  @Patch('/:id')
  async update(
    @Param('id', ParseIntPipe) id: number,
    @Body() data: unknown,
  ): Promise<unknown> {
    return this.kategoriService.update(id, data);
  }

  @Delete('/:id')
  async remove(@Param('id', ParseIntPipe) id: number): Promise<unknown> {
    return this.kategoriService.remove(id);
  }
}
```

Controller di gateway hanya berfungsi sebagai penerima request dari user, lalu langsung diteruskan ke proxy service tanpa validasi atau logic tambahan (validasi tetap dilakukan oleh service aslinya).

Bagian	Fungsi
@Controller('kategori')	Semua endpoint di file ini otomatis diawali /api/kategori
@Body() data: unknown	Data dari request body diteruskan langsung apa adanya ke service asli, validasi tetap dilakukan di kategori-service
@Param('id', ParseIntPipe)	Parameter id dikonversi ke number sebelum diteruskan

File product.controller.ts, order.controller.ts, dan inventory.controller.ts memiliki struktur yang sama persis, hanya beda nama class dan path.

5. api/gateway/src/kategori/kategori.module.ts

```
import { Module } from '@nestjs/common';
import { HttpModule } from '@nestjs/axios';
```

```
import { KategoriService } from '../kategori.service';
import { KategoriController } from '../kategori.controller';

@Module({
  imports: [HttpModule],
  controllers: [KategoriController],
  providers: [KategoriService],
})
export class KategoriModule {}
```

Module ini mendaftarkan HttpModule agar KategoriService bisa menggunakan HttpService untuk melakukan request ke service lain.

Bagian	Fungsi
imports: [HttpModule]	Menyediakan HttpService dari @nestjs/axios agar bisa di-inject ke KategoriService
controllers, providers	Sama seperti module pada service lain, mendaftarkan controller dan service

File product.module.ts, auth.module.ts, order.module.ts, dan inventory.module.ts memiliki struktur yang sama persis, hanya beda nama class.

6. api/gateway/src/app.module.ts

```
import { Module } from '@nestjs/common';
import { KategoriModule } from '../kategori/kategori.module';
import { ProductModule } from '../product/product.module';
import { AuthModule } from '../auth/auth.module';
import { OrderModule } from '../order/order.module';
import { InventoryModule } from '../inventory/inventory.module';

@Module({
  imports: [
    KategoriModule,
    ProductModule,
    AuthModule,
    OrderModule,
    InventoryModule,
  ],
  controllers: [],
  providers: [],
})
export class AppModule {}
```

Module utama gateway yang mendaftarkan seluruh module proxy, sehingga semua endpoint dari 5 microservice bisa diakses melalui satu pintu masuk (port 3000).

Bagian	Fungsi
--------	--------

KategoriModule, ProductModule, AuthModule, OrderModule, InventoryModule	Module proxy untuk masing-masing microservice yang sudah dibuat sebelumnya
controllers: [], providers: []	Kosong karena setiap module proxy sudah memiliki controller dan provider sendiri

7. api/gateway/src/main.ts

```
import 'dotenv/config';
import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { AppModule } from '../app.module';

async function bootstrap(): Promise<void> {
  const app = await NestFactory.create(AppModule);
  app.setGlobalPrefix('api');
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    }),
  );
  await app.listen(3000);
}

bootstrap();
```

Titik masuk utama gateway. Sama seperti main.ts pada service lain, dengan tambahan import 'dotenv/config' agar variabel KATEGORI_SERVICE_URL dan sejenisnya dari .env terbaca saat runtime.

Bagian	Fungsi
import 'dotenv/config'	Memastikan semua variabel di .env (URL setiap service) terbaca sebelum proxy service dibuat
app.setGlobalPrefix('api')	Semua endpoint diawali /api/, sama seperti service lain
app.listen(3000)	Gateway berjalan di port 3000 sebagai entry point utama, port 3001-3005 dipakai oleh masing-masing microservice

Dengan demikian, semua request dari user (misal <http://localhost:3000/api/kategori>) akan diteruskan oleh gateway ke service aslinya (<http://localhost:3001/api/kategori>), lalu hasilnya dikirim balik ke user.